Page: Details | Result: 0 - 1616 - ampere_sgemm_64x64_nr | Add Baseline | Apply Rules | Occupancy Calculator | Source Comparison | Save as PDF

	Result	Time	Cycles	Regs	GPU	SM Frequency	CC	Process
Current	1616 - ampere_sgemm_64x64_nn (32, 32, 1)x(64, 1, ...	2.89 msecond	46,74,835	126	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.62 cycle/nsecond	8.9	[131688] gemm_gpu

GPU Speed Of Light Throughput

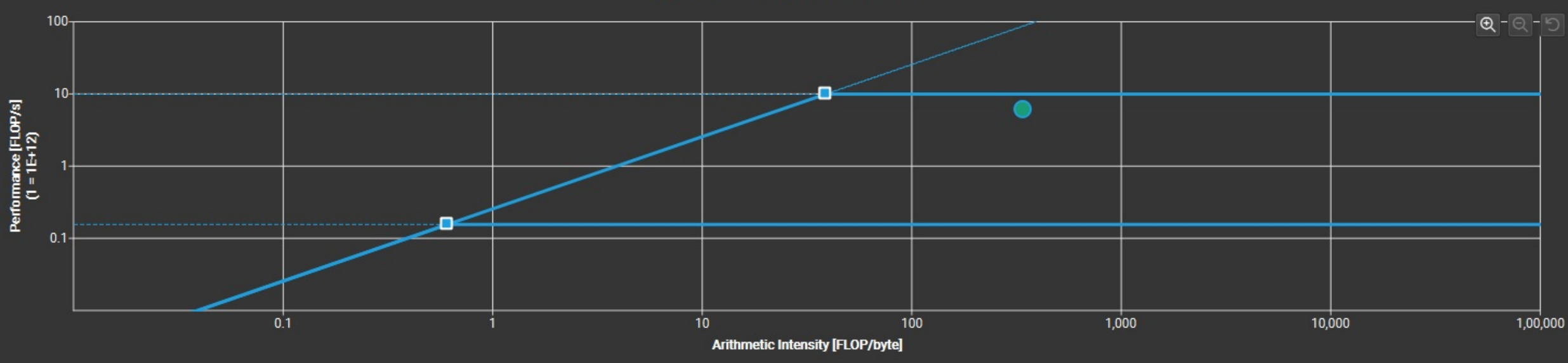
GPU Throughput Rooflines

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	72.02	Duration [msecond]	2.89
Memory Throughput [%]	60.59	Elapsed Cycles [cycle]	46,74,835
L1/TEX Cache Throughput [%]	62.41	SM Active Cycles [cycle]	45,38,033.96
L2 Cache Throughput [%]	28.07	SM Frequency [cycle/nsecond]	1.62
DRAM Throughput [%]	6.80	DRAM Frequency [cycle/nsecond]	7.99

High Compute Throughput Compute is more heavily utilized than Memory: Look at the Compute Workload Analysis section to see what the compute pipelines are spending their time doing. Also, consider whether any computation is redundant and could be reduced or moved to look-up tables.

Roofline Analysis The ratio of peak float (fp32) to double (fp64) performance on this device is 64:1. The kernel achieved 60% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the Kernel Profiling Guide for more details on roofline analysis.



PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand workload behavior changes over its runtime.

Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	2.88	SM Busy [%]	74.19
Executed Ipc Active [inst/cycle]	2.97	Issue Slots Busy [%]	74.19
Issued Ipc Active [inst/cycle]	2.97		

High Utilization FMA is the highest-utilized pipeline (61.7%) based on active cycles, taking into account the rates of its different instructions. It executes 32-bit floating point (FADD, FMUL, FMAD, ...) and integer (IMUL, IMAD) operations. The pipeline is well-utilized, but might become a bottleneck if more work is added. Based on the number of executed instructions, the highest utilized pipeline (62.4%) is LSU. It executes load/store memory operations. Comparing the two, the overall pipeline utilization appears to be caused by frequent, low-latency instructions. See the Kernel Profiling Guide or hover over the pipeline name to understand the workloads handled by each pipeline. The Instruction Statistics section shows the mix of executed instructions in this kernel. Check the Warp State Statistics section for which reasons cause warps to stall.

Memory Workload Analysis

Memory Chart

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/second]	17.41	Mem Busy [%]	58.31
L1/TEX Hit Rate [%]	3.16	Max Bandwidth [%]	60.59
L2 Hit Rate [%]	98.12	Mem Pipes Busy [%]	60.59
L2 Compression Success Rate [%]	0	L2 Compression Ratio	0

L2 Load Access Pattern **Est. Speedup: 13.38%** The memory access pattern for loads from L1TEX to L2 is not optimal. The granularity of an L1TEX request to L2 is a 128 byte cache line. That is 4 consecutive 32-byte sectors per L2 request. However, this kernel only accesses an average of 1.6 sectors out of the possible 4 sectors per cache line. Check the Source Counters section for uncoalesced loads and try to minimize how many cache lines need to be accessed per memory request.

Shared Store Bank Conflicts **Est. Speedup: 6.38%** The memory access pattern for shared stores might not be optimal and causes on average a 1.1 - way bank conflict across all 8423424 shared store requests.This results in 970751 bank conflicts, which represent 10.23% of the overall 9492651 wavefronts for shared stores. Check the Source Counters section for uncoalesced shared stores.

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	3.81	No Eligible [%]	25.66
Eligible Warps Per Scheduler [warp]	2.13	One or More Eligible [%]	74.34
Issued Warp Per Scheduler	0.74		

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	5.13	Avg. Active Threads Per Warp	32
Warp Cycles Per Executed Instruction [cycle]	5.13	Avg. Not Predicated Off Threads Per Warp	31.92

Not Selected Stalls **Est. Local Speedup: 36.39%** On average, each warp of this kernel spends 1.9 cycles being stalled waiting for the micro scheduler to select the warp to issue. Not selected warps are eligible warps that were not picked by the scheduler to issue that cycle as another warp was selected. A high number of not selected warps typically means you have sufficient warps to cover warp latencies and you may consider reducing the number of active warps to possibly increase cache coherence and data locality. This stall type represents about 36.4% of the total average of 5.1 cycles between issuing two instructions.

Warp Stall Check the Warp Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The Kernel Profiling Guide provides more details on each stall reason.

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	32,32,01,024	Avg. Executed Instructions Per Scheduler [inst]	33,66,677.33
Issued Instructions [inst]	32,32,05,731	Avg. Issued Instructions Per Scheduler [inst]	33,66,726.36

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	1,024	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	126	Static Shared Memory Per Block [Kbyte/block]	8.45
Block Size	64	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	65,536	Driver Shared Memory Per Block [Kbyte/block]	1.02
Waves Per SM	5.33	Shared Memory Configuration Size [Kbyte]	102.40

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	33.33	Block Limit Registers [block]	8
Theoretical Active Warps per SM [warp]	16	Block Limit Shared Mem [block]	10
Achieved Occupancy [%]	31.71	Block Limit Warps [block]	24
Achieved Active Warps Per SM [warp]	15.22	Block Limit SM [block]	24

Theoretical Occupancy **Est. Local Speedup: 66.67%** The 4.00 theoretical warps per scheduler this kernel can issue according to its occupancy are below the hardware maximum of 12. This kernel's theoretical occupancy (33.3%) is limited by the number of required registers.

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	6,28,736	Branch Efficiency [%]	100
Branch Instructions Ratio [%]	0.00	Avg. Divergent Branches	0

Uncoalesced Shared Accesses **Est. Speedup: 0.30%** This kernel has uncoalesced shared accesses resulting in a total of 131072 excessive wavefronts (0% of the total 42403840 wavefronts). Check the L1 Wavefronts Shared Excessive table for the primary source locations. The CUDA Best Practices Guide has an example on optimizing shared memory accesses.

L1 Wavefronts Shared Excessive

Location	Value	Value (%)
0x500e76f90 in ampere_sgemm_64x64_nn	65,536	50
0x500e76f80 in ampere_sgemm_64x64_nn	65,536	50
0x500e77150 in ampere_sgemm_64x64_nn	0	0
0x500e77140 in ampere_sgemm_64x64_nn	0	0
0x500e77130 in ampere_sgemm_64x64_nn	0	0

Follow the *rules outputs* to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel.
You could also disable individual sections to focus on selected performance aspects and make profiling faster.